

Inverted Index and Search with Hadoop and Spark

Cloud Computing Project Report

Reza Almassi – Ronald Omoding – Rojan Shrestha

Academic Year 2024/2025

Source code: <https://github.com/Rezacs/Cloud>

1 Project Overview

The goal of the project was to build a basic search-engine backend based on an *inverted index*. For each word detected in a collection of text files, the output stores the list of filenames where the word appears and its frequency inside each file, for example:

```
cloud doc1.txt:3 doc2.txt:1
```

We implemented and evaluated five solutions:

1. **Hadoop Base** in Java, using a standard combiner.
2. **Hadoop InMapper** in Java, using in-mapper combining.
3. **PySpark** (Spark Optimized), in Python.
4. **Java Spark**, an equivalent Spark implementation in Java.
5. **Sequential Python**, used as a single-machine baseline.

The distributed Hadoop and Spark methods read input from **HDFS**. The sequential Python solution reads from the **local filesystem**. This is intentional: the baseline uses normal Python APIs such as `os.walk()` and `open()` and is interpreted as a local single-machine reference rather than a cluster job.

1.1 Cluster and Software

The project was deployed on a three-node fully distributed Hadoop cluster using **Hadoop 3.1.3**, **Java 8**, and **Apache Spark**. Each VM had 2 CPU cores and 6.8 GB of RAM. The main node also participated as a worker, running both master-side and worker-side YARN daemons.

Table 1: Cluster node roles.

Node	Role	IP address
namenode	NameNode, SecondaryNameNode, ResourceManager, DataNode, NodeManager	10.1.1.166
datanode2	DataNode, NodeManager	10.1.1.213
datanode3	DataNode, NodeManager	10.1.1.163

HDFS was configured with replication factor 2. YARN was configured with three active NodeManagers, so all three machines were eligible to execute distributed tasks.

2 Dataset Preparation

2.1 Source Collections

The original source files were stored under `/home/hadoop/Cloud/source_downloads/` and came from three collections:

- **gutenberg_big** — public-domain books from Project Gutenberg.
- **archive_big** — text files from the Internet Archive.
- **kaggle_news** — news articles from the Kaggle News Articles Corpus dataset: <https://www.kaggle.com/datasets/sbhatti/news-articles-corpus>.

2.2 General Preparation Method

Most datasets were built by selecting files from the source folders using a *round-robin* strategy, taking files alternately from the available collections until a target size was reached (measured in binary megabytes: 1 MB = 1024×1024 bytes). Files were *moved* rather than copied in order to avoid duplication and track remaining source stock. Some later datasets were built by copying or combining existing prepared datasets to create specific combined workloads.

2.3 Individual Dataset Construction

- **ds-00** (100 files, 0.43 MB) and **ds-01** (1,000 files, 4.14 MB): pre-prepared small news datasets.
- **ds-02** (2,582 files, 261 MB), **ds-04** (844 files, 339 MB), **ds-05** (807 files, 500 MB), **ds-07** (2,493 files, 800 MB), **ds-08** (6,680 files, 1126.40 MB): round-robin selection from all three source collections.
- **ds-03** (642 files, 297.89 MB): round-robin from **gutenberg_big** and **archive_big** only (excludes **kaggle_news**).
- **ds-06** (266 files, 752.03 MB): all remaining **gutenberg_big** files.
- **ds-09** (18,680 files, 1173.86 MB): a copy of ds-08 plus 12,000 additional files from **kaggle_news**.

- **ds-10** (1,917 files, 1591.03 MB): a combination of ds-04, ds-05, and ds-06.
- **ds-11** (116 files, 96 MB): files from the Internet Archive collection.
- **ds-12** (2,495 files, 1126.40 MB): files from Project Gutenberg.

After construction, `gutenberg_big` and `archive_big` were fully consumed; approximately 850,638 files (≈ 3.2 GB) remained unused in `kaggle_news`.

2.4 Combined Datasets

In addition to the 13 individually prepared datasets, 12 *combined* datasets were created by passing multiple comma-separated HDFS paths directly to Hadoop and Spark jobs, without physically copying or merging any files on HDFS. Both `FileInputFormat` (Hadoop) and `wholeTextFiles()` (Spark) accept multiple input directories natively. The sequential Python baseline, which has no multi-path support, instead copies each path into its own subfolder of a temporary local directory and walks the parent recursively.

Table 2: All 25 datasets used in the final experiments, ordered by approximate size.

Dataset	Composition	Files	Approx. Size
ds-00-000mb-100files-tiny-news	base	100	0.43 MB
ds-01-004mb-1000files-news-small	base	1,000	4.14 MB
combo-06-ds00-01	ds-00 + ds-01	1,100	4.57 MB
ds-11-096mb-116files-archive	base	116	96 MB
ds-03-298mb-642files	base	642	297.89 MB
ds-02-261mb-2582files	base	2,582	261 MB
ds-04-339mb-844files	base	844	339 MB
ds-05-500mb-807files	base	807	500 MB
combo-01-ds02-03	ds-02 + ds-03	3,224	559 MB
ds-06-752mb-266files-gutenberg-remaining	base	266	752.03 MB
ds-07-800mb-2493files	base	2,493	800 MB
combo-02-ds06-11	ds-06 + ds-11	382	848 MB
ds-08-1p1gb-6680files	base	6,680	1126.40 MB
ds-12-1p1gb-2495files-gutenberg	base	2,495	1126.40 MB
combo-07-ds03-04-05	ds-03+ds-04+ds-05	2,293	1137 MB
ds-09-1p15gb-18680files	base	18,680	1173.86 MB
combo-04-ds01-09	ds-01 + ds-09	19,680	1178 MB
combo-08-ds02-09	ds-02 + ds-09	21,262	1435 MB
ds-10-1p55gb-1917files-combined	base	1,917	1591.03 MB
combo-09-ds06-07-11	ds-06+ds-07+ds-11	2,875	1648 MB
combo-03-ds07-12	ds-07 + ds-12	4,988	1926 MB
combo-05-ds09-10	ds-09 + ds-10	20,597	2765 MB
combo-11-ds06-07-10	ds-06+ds-07+ds-10	4,676	3142 MB
combo-10-ds08-09-12	ds-08+ds-09+ds-12	27,855	3426 MB
combo-12-ds08-10-12	ds-08+ds-10+ds-12	11,092	3843 MB

3 Implementation

3.1 Common Processing Logic

All implementations share the same tokenization rule (`[a-z0-9]+`), lowercase conversion, and stop-word removal. The output format is:

```
word    filename1:count1 filename2:count2 ...
```

3.2 Hadoop Base with Combiner

The Mapper emits one key-value pair per non-stopword token, keyed as `word@filename`. A Combiner sums equal keys before the shuffle, and the Reducer converts all `word@filename` pairs into one posting list per word. A custom `WholeFileInputFormat` reads each file as a single record using the filename as the map key, which is necessary to preserve original filenames in the output.

Listing 1: Hadoop Base: mapper output and combiner.

```
// Mapper
while (matcher.find()) {
    if (!stopWords.contains(matcher.group())) {
        outputKey.set(matcher.group() + "@" + filename);
        context.write(outputKey, one);
    }
}

// Combiner
int sum = 0;
for (IntWritable v : values) sum += v.get();
context.write(key, new IntWritable(sum));
```

3.3 Hadoop InMapper Combining

The InMapper variant accumulates word counts in a per-file `HashMap` inside the mapper and emits only one record per unique `word@filename` pair, eliminating repeated intermediate records for the same word within a file and directly reducing shuffle volume.

Listing 2: Hadoop InMapper: local aggregation.

```

localCounts.clear();
while (matcher.find()) {
    String word = matcher.group();
    if (!stopWords.contains(word)) {
        String k = word + "@" + filename;
        localCounts.put(k, localCounts.getOrDefault(k, 0) + 1);
    }
}
for (Map.Entry<String,Integer> e : localCounts.entrySet())
    context.write(new Text(e.getKey()), new IntWritable(e.getValue()));

```

3.4 PySpark (Spark Optimized)

The PySpark solution uses `wholeTextFiles()` so each file is processed as a single document. Words are counted locally per file, then `combineByKey` aggregates posting lists across partitions. The global `sortByKey()` was removed because it introduced an additional wide shuffle with no benefit for the inverted index itself; postings within each word's list are still sorted locally.

Listing 3: PySpark core pipeline.

```

files = sc.wholeTextFiles(input_path, minPartitions=num_partitions * 2)
inverted_index = (
    files
    .flatMap(file_to_postings) # local per-file word counting
    .combineByKey(create_combiner,
                  merge_value,
                  merge_combiners,
                  numPartitions=num_partitions)
    .mapValues(lambda postings: " ".join(sorted(postings)))
)
inverted_index.map(lambda x: f"{x[0]}\t{x[1]}").saveAsTextFile(output_path)

```

Additional optimizations enabled: Kryo serialization, shuffle compression, RDD compression, and Python worker reuse (`spark.python.worker.reuse=true`).

3.5 Java Spark

A second Spark implementation was written in Java, using identical algorithmic structure to PySpark (per-file local counting, `combineByKey`, no global sort) so that language and runtime overhead can be isolated from algorithmic behaviour. Stop-words are broadcast to all executors via `JavaSparkContext.broadcast()`.

Listing 4: Java Spark: `flatMapToPair` and `combineByKey`.

```

JavaPairRDD<String,String> postings = files.flatMapToPair(file -> {
    String filename = file._1().substring(file._1().lastIndexOf("/") + 1);
    Map<String,Integer> counts = new HashMap<>();
    Matcher m = TOKEN_RE.matcher(file._2());
    while (m.find()) {
        String w = m.group().toLowerCase();
        if (!stopwordsBC.value().contains(w))
            counts.put(w, counts.getOrDefault(w, 0) + 1);
    }
    List<Tuple2<String,String>> out = new ArrayList<>();
    for (Map.Entry<String,Integer> e : counts.entrySet())
        out.add(new Tuple2<>(e.getKey(), filename + ":" + e.getValue()));
    return out.iterator();
});

JavaPairRDD<String,List<String>> index = postings.combineByKey(
    v -> { List<String> l = new ArrayList<>(); l.add(v); return l; },
    (l,v) -> { l.add(v); return l; },
    (a,b) -> { a.addAll(b); return a; },
    partitions
);
index.map(t -> { Collections.sort(t._2());
    return t._1() + "\t" + String.join(" ", t._2()); })
    .saveAsTextFile(output);

```

3.6 Spark Cluster Configuration

Both Spark engines were submitted with the same fixed YARN configuration for every dataset, removing executor memory as a confounding variable between datasets.

Table 3: Spark submit configuration, applied uniformly to all 25 datasets and both Spark engines.

Parameter	Value
-master / -deploy-mode	yarn / client
-num-executors	3
-executor-memory	3584 m
-executor-cores	2
spark.executor.memoryOverhead	512 m
-driver-memory	2 g (on namenode, outside YARN cap)
spark.yarn.am.memory	512 m
spark.dynamicAllocation.enabled	false
spark.hadoop.mapreduce.input.fileinputformat.list-status.num-threads	8
spark.network.timeout	600 s
spark.executor.heartbeatInterval	60 s

Each executor requests $3584 + 512 = 4096$ MB; three executors plus the Application Master ($512 + \approx 384$ MB overhead) total approximately 13.2 GB out of the cluster’s 16.2 GB YARN capacity. Setting `executor-cores=2` allows each executor JVM to use both physical cores of its node, replacing an earlier single-core configuration.

3.7 Sequential Python and Search

The sequential baseline builds the inverted index in a Python `defaultdict` and writes output in sorted word order from local files only.

Listing 5: Sequential Python core loop.

```

inverted_index = defaultdict(dict)
for filepath in collect_files(input_path):
    filename = os.path.basename(filepath)
    with open(filepath, "r", encoding="utf-8", errors="replace") as f:
        text = f.read()
    counts = Counter(w for w in tokenize(text) if w not in stopwords)
    for word, count in counts.items():
        inverted_index[word][filename] = count

```

A non-parallel search script loads the generated index and answers single-term or multi-term queries, returning the intersection of filename sets for multi-term queries.

4 Experimental Methodology

4.1 Cluster Resource Configuration

Table 4: Per-node and cluster-wide YARN resource configuration.

Parameter	Value
RAM per node (physical)	6.8 GB
CPU cores per node (physical)	2
yarn.nodemanager.resource.memory-mb	5,400 MB
yarn.nodemanager.resource.cpu-vcores	3
yarn.scheduler.maximum-allocation-mb	5,400 MB
Cluster-wide YARN memory (3×5400)	16,200 MB
Cluster-wide YARN vcores (3×3)	9
HDFS replication factor	2

The configured 3 vcores per node slightly exceeds the 2 physical cores available; this pre-existing oversubscription was left unchanged.

4.2 Parallelism Parameters

For **Hadoop** (both Base and InMapper), the reducer count was swept as $r \in \{1, 2, 4, 8, 16, 24\}$. Hadoop’s disk-backed shuffle architecture does not suffer from single-reducer memory collapse, so $r = 1$ remained a valid and informative measurement across all 25 datasets.

For **Spark** (both PySpark and Java Spark), the partition count was swept as $p \in \{4, 8, 16, 24, 32, 40\}$. The values $p = 1$ and $p = 2$ used in earlier runs were removed because they consistently caused either immediate job failure (`exit status 1`, no output) or runtimes of 30–50 minutes on datasets larger than ≈ 1 GB: with a single partition, the entire dataset must be held in one executor’s heap during garbage collection, which exhausts the available memory. The values $p = 32$ and $p = 40$ were added to determine whether performance continued to improve beyond the $p = 24$ plateau observed in earlier runs.

4.3 Measurement Procedure

Each job was launched under `/usr/bin/time -v` to record wall-clock elapsed time and the peak resident set size (RSS) of the client/driver process. A background monitor sampled `yarn node -status` and `free -m` on all three nodes every 5 seconds throughout each job, recording per-node YARN-allocated memory and per-node OS memory usage. The per-node

samples were summed across all three nodes at each timestamp; the maximum across all timestamps was recorded as the job’s peak cluster-wide system memory (`system_used_max_gb`) and peak YARN allocation (`max_yarn_allocated_gb`). HDFS output was deleted and `/tmp` was cleared on all three nodes between every individual job run, ensuring no residual data could affect subsequent measurements. Correctness was validated by counting output lines for every job.

4.4 Anomalies Encountered and Resolved

ds-09 nested HDFS directory. In an earlier run, all Hadoop jobs on `ds-09` produced zero output lines despite exit status 0, and all Spark jobs failed immediately after 30–50 seconds. Inspection revealed that all 18,680 files were nested one directory level deeper than expected due to how the dataset was uploaded. Hadoop’s `FileInputFormat` does not recurse into subdirectories by default, so it found zero input splits and produced a technically successful but empty result; Spark raised an explicit “0 files matched” error at job-planning time. The fix was a one-time `hdfs dfs -mv` to flatten the directory. After the fix, all five methods produced the expected output.

combo-11 sequential Python out-of-memory. The sequential Python run on `combo-11` (3.1 GB) was terminated by the OS out-of-memory killer (exit status 137) after 423 seconds. Holding the full in-memory dictionary for 3.1 GB of input on a single machine with no spill-to-disk mechanism exceeded available RAM on the `namenode`. All four distributed methods completed `combo-11` successfully.

5 Results

Out of 625 individual job executions, 624 completed successfully. The one exception (`combo-11` sequential Python, OOM) is excluded from the tables below.

5.1 Best Elapsed Time per Dataset

Table 5: Best elapsed time (seconds) per dataset and method. r = reducer count for Hadoop; p = partition count for Spark. Sequential Python is a single local run with no parameter. -- = OOM failure.

Dataset	Size (MB)	H.Base	H.InMap	PySpark	J.Spark	Seq.Py
ds-00 (0.43 MB, 100 files)	0.43	29	25	41	39	0
ds-01 (4.14 MB, 1000 files)	4.14	35	32	42	42	1
combo-06 (4.57 MB, 1100 files)	4.57	33	31	42	40	2
ds-11 (96 MB, 116 files)	96	73	45	48	46	17
ds-03 (297.89 MB, 642 files)	297.89	119	70	61	61	52
ds-02 (261 MB, 2582 files)	261	127	78	62	62	48
ds-04 (339 MB, 844 files)	339	116	67	59	63	58
ds-05 (500 MB, 807 files)	500	98	66	70	72	84
combo-01 (559 MB, 3224 files)	559	128	82	81	78	100
ds-06 (752 MB, 266 files)	752.03	113	83	100	94	108
ds-07 (800 MB, 2493 files)	800	119	75	86	86	144
combo-02 (848 MB, 382 files)	848	115	85	90	96	126
ds-08 (1126 MB, 6680 files)	1126.40	167	105	95	86	172
ds-12 (1126 MB, 2495 files)	1126.40	154	97	97	81	167
combo-07 (1137 MB, 2293 files)	1137	183	111	112	112	207
ds-09 (1174 MB, 18680 files)	1173.86	208	152	107	105	188
combo-04 (1178 MB, 19680 files)	1178	171	115	112	107	194
combo-08 (1435 MB, 21262 files)	1435	261	143	143	125	270
ds-10 (1591 MB, 1917 files)	1591.03	238	138	151	133	265
combo-09 (1648 MB, 2875 files)	1648	231	144	139	133	281
combo-03 (1926 MB, 4988 files)	1926	267	160	141	128	356
combo-05 (2765 MB, 20597 files)	2765	458	314	218	211	680
combo-11 (3142 MB, 4676 files)	3142	428	243	239	227	—
combo-10 (3426 MB, 27855 files)	3426	479	295	237	212	840
combo-12 (3843 MB, 11092 files)	3843	498	345	265	245	805

5.2 Best Parameter per Method

Table 6: Parameter value achieving the best elapsed time, selected representative datasets.

Dataset	Hadoop Base	Hadoop InMapper	PySpark	Java Spark
ds-00 (0.43 MB)	$r = 4$	$r = 2$	$p = 8$	$p = 4$
ds-11 (96 MB)	$r = 2$	$r = 4$	$p = 8$	$p = 24$
ds-02 (261 MB)	$r = 2$	$r = 2$	$p = 24$	$p = 24$
ds-05 (500 MB)	$r = 2$	$r = 4$	$p = 16$	$p = 40$
ds-08 (1126 MB)	$r = 4$	$r = 8$	$p = 16$	$p = 32$
ds-09 (1174 MB)	$r = 8$	$r = 8$	$p = 24$	$p = 32$
ds-10 (1591 MB)	$r = 16$	$r = 2$	$p = 16$	$p = 32$
combo-03 (1926 MB)	$r = 8$	$r = 2$	$p = 40$	$p = 40$
combo-05 (2765 MB)	$r = 2$	$r = 16$	$p = 32$	$p = 40$
combo-10 (3426 MB)	$r = 2$	$r = 1$	$p = 40$	$p = 24$
combo-12 (3843 MB)	$r = 2$	$r = 2$	$p = 32$	$p = 32$

Hadoop InMapper achieves its best time at low reducer counts ($r = 1-4$ in most cases), since the bulk of its work is already done in the map phase. Both Spark engines tend to improve with higher partition counts as dataset size grows, typically settling between $p = 24$ and $p = 40$ on datasets above 1 GB. Hadoop Base shows no single consistently best value, indicating its bottleneck is shuffle volume rather than reducer parallelism. A detailed discussion of this pattern is given in Section 7.

5.3 Memory Usage

Memory is reported using two complementary metrics:

- **system_used_max_gb**: peak combined OS memory across all three nodes during a job (summed from 5-second samples). Reflects real cluster-wide memory pressure.
- **Driver RSS (MB)**: peak resident set size of the client/driver process on **namenode** (from `/usr/bin/time -v`). Reflects client-side overhead only.

The YARN allocation figure (**max_yarn_allocated_gb**) is not used as a primary metric because it is capped at approximately 4–5 GB by the fixed executor configuration regardless of dataset size, and therefore does not reflect workload variation.

Table 7: Peak combined system memory across all three nodes (GB), mean over all parameter values, per dataset and method. -- = OOM.

Dataset	H.Base	H.InMap	PySpark	J.Spark	Seq.Py
ds-00 (0.43 MB)	2.76	2.88	3.03	3.03	0.00
ds-01 (4.14 MB)	2.77	2.74	3.17	3.06	0.00
combo-06 (4.57 MB)	2.89	2.64	3.20	3.09	0.00
ds-11 (96 MB)	3.14	3.24	3.24	3.20	2.45
ds-02 (261 MB)	3.82	3.82	3.32	3.66	2.57
ds-03 (297.89 MB)	3.87	3.88	3.68	3.84	2.74
ds-04 (339 MB)	3.88	3.95	3.49	3.70	2.81
ds-05 (500 MB)	3.21	3.49	3.56	4.08	3.19
combo-01 (559 MB)	3.86	3.95	3.55	4.29	3.11
ds-06 (752 MB)	4.08	4.08	3.80	4.09	3.64
ds-07 (800 MB)	4.07	4.05	3.82	4.49	3.87
combo-02 (848 MB)	4.11	3.98	3.96	4.26	3.78
ds-08 (1126 MB)	3.92	3.77	3.67	4.30	2.94
ds-12 (1126 MB)	3.79	3.78	3.69	4.33	3.25
combo-07 (1137 MB)	3.87	3.92	3.92	4.56	4.12
ds-09 (1174 MB)	4.01	4.33	4.06	4.73	3.04
combo-04 (1178 MB)	3.99	3.92	3.97	4.73	3.04
combo-08 (1435 MB)	3.94	3.93	4.36	4.85	3.54
ds-10 (1591 MB)	4.63	4.88	4.32	4.77	5.03
combo-09 (1648 MB)	4.11	4.24	4.58	4.70	5.35
combo-03 (1926 MB)	4.00	3.97	4.05	4.64	4.90
combo-05 (2765 MB)	4.79	4.87	4.91	5.63	5.89
combo-11 (3142 MB)	4.21	4.31	5.02	5.54	—
combo-10 (3426 MB)	4.22	4.19	4.70	5.43	4.09
combo-12 (3843 MB)	4.58	4.96	4.69	5.62	6.33

Table 8: Peak driver/client process RSS (MB), mean over all parameter values, per dataset and method. For sequential Python this is the entire computation, not just submission overhead.

Dataset	H.Base	H.InMap	PySpark	J.Spark	Seq.Py
ds-00 (0.43 MB)	321	336	536	514	13
ds-01 (4.14 MB)	344	327	557	547	30
combo-06 (4.57 MB)	326	329	572	524	30
ds-11 (96 MB)	334	333	568	516	357
ds-02 (261 MB)	331	325	592	502	578
ds-03 (297.89 MB)	331	342	577	530	714
ds-04 (339 MB)	318	316	549	516	760
ds-05 (500 MB)	320	328	560	531	1,238
combo-01 (559 MB)	343	326	597	533	1,156
ds-06 (752 MB)	332	338	559	514	1,929
ds-07 (800 MB)	321	341	570	552	1,990
combo-02 (848 MB)	320	322	568	509	2,109
ds-08 (1126 MB)	376	362	662	594	933
ds-12 (1126 MB)	313	329	578	532	1,299
combo-07 (1137 MB)	311	345	559	510	2,189
ds-09 (1174 MB)	556	526	858	768	1,056
combo-04 (1178 MB)	541	583	880	826	1,062
combo-08 (1435 MB)	575	530	914	856	1,572
ds-10 (1591 MB)	327	330	563	522	3,181
combo-09 (1648 MB)	331	322	554	531	3,736
combo-03 (1926 MB)	353	337	610	552	3,144
combo-05 (2765 MB)	543	560	880	813	4,005
combo-11 (3142 MB)	341	338	558	561	4,751
combo-10 (3426 MB)	640	634	987	967	2,111
combo-12 (3843 MB)	446	436	668	688	4,490

6 Runtime Comparison Plots

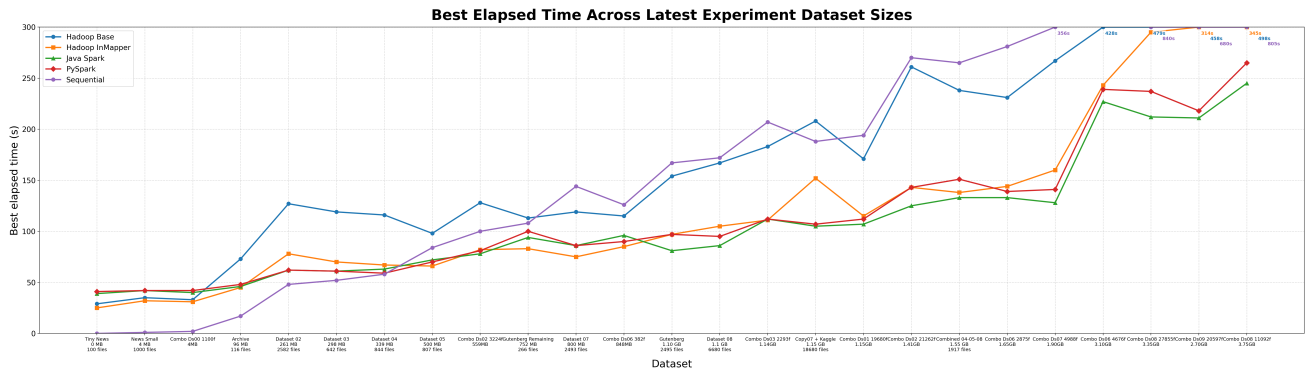


Figure 1 — Best elapsed time vs. dataset size (all 25 datasets, 5 methods).

X-axis: dataset size in MB (log scale, 0.43 to 3843). Y-axis: best elapsed time in seconds. One line per method. Shows the crossover above which both Spark engines overtake Hadoop InMapper, and the point above which Sequential Python becomes the slowest method before failing entirely.

Figure 2 — Spark Partition Sweep: Elapsed Time vs p

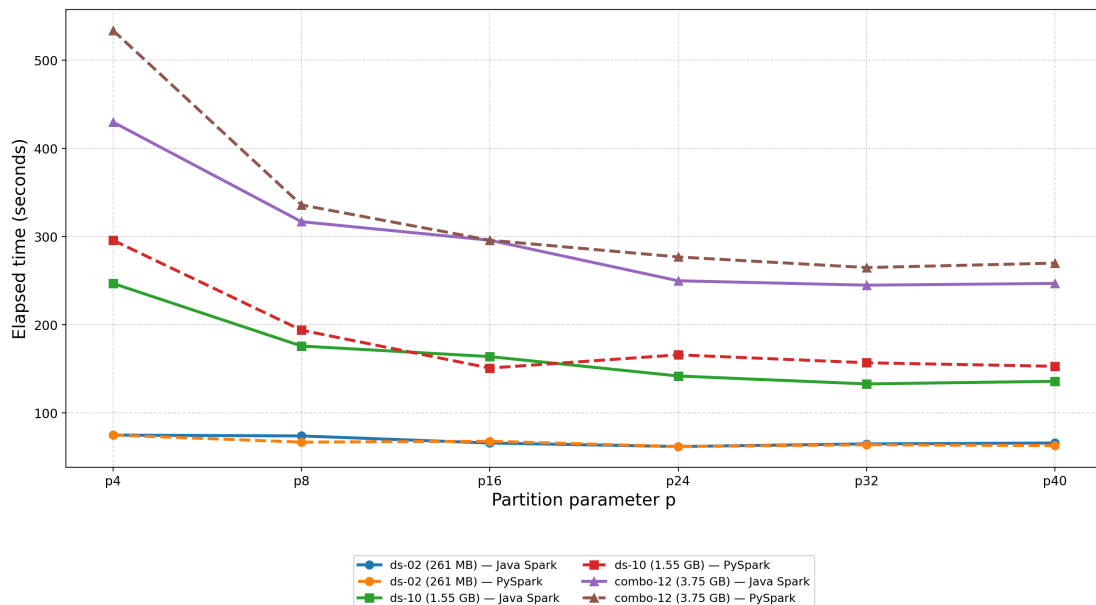


Figure 2 — Spark partition sweep: elapsed time vs. p for PySpark and Java Spark.

Three representative sizes: ds-02 (≈ 261 MB), ds-10 (≈ 1591 MB), combo-12 (≈ 3843 MB). X-axis: $p \in \{4, 8, 16, 24, 32, 40\}$. Shows the plateau beyond $p = 24-32$ and the growing gap between Java Spark and PySpark at larger scale.

Figure 3 — Hadoop Reducer Sweep: Elapsed Time vs r

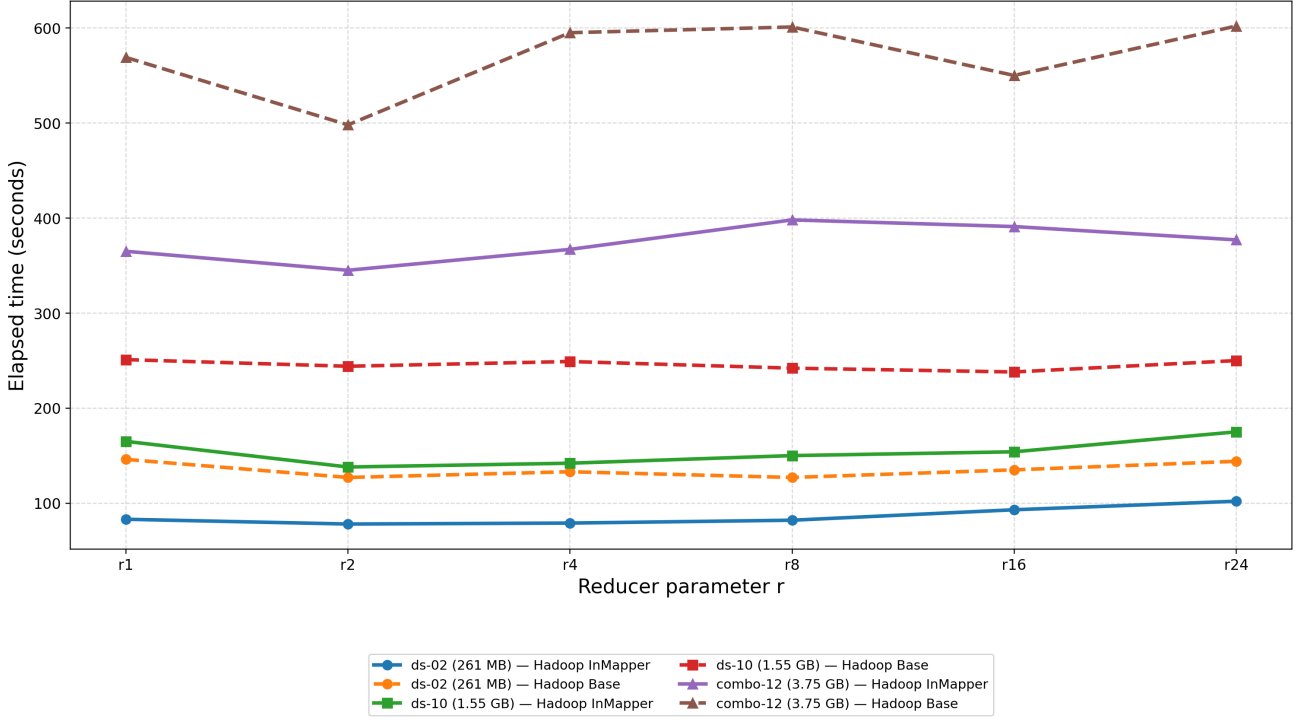


Figure 3 — Hadoop reducer sweep: elapsed time vs. r for Hadoop Base and InMapper.

Same three representative sizes as Figure 2. X-axis: $r \in \{1, 2, 4, 8, 16, 24\}$. Shows InMapper consistently below Base at every reducer count, with its optimum typically at $r = 1-4$.

7 Memory Usage Plots

Figure 4 — Peak Cluster-wide System Memory vs Dataset Size

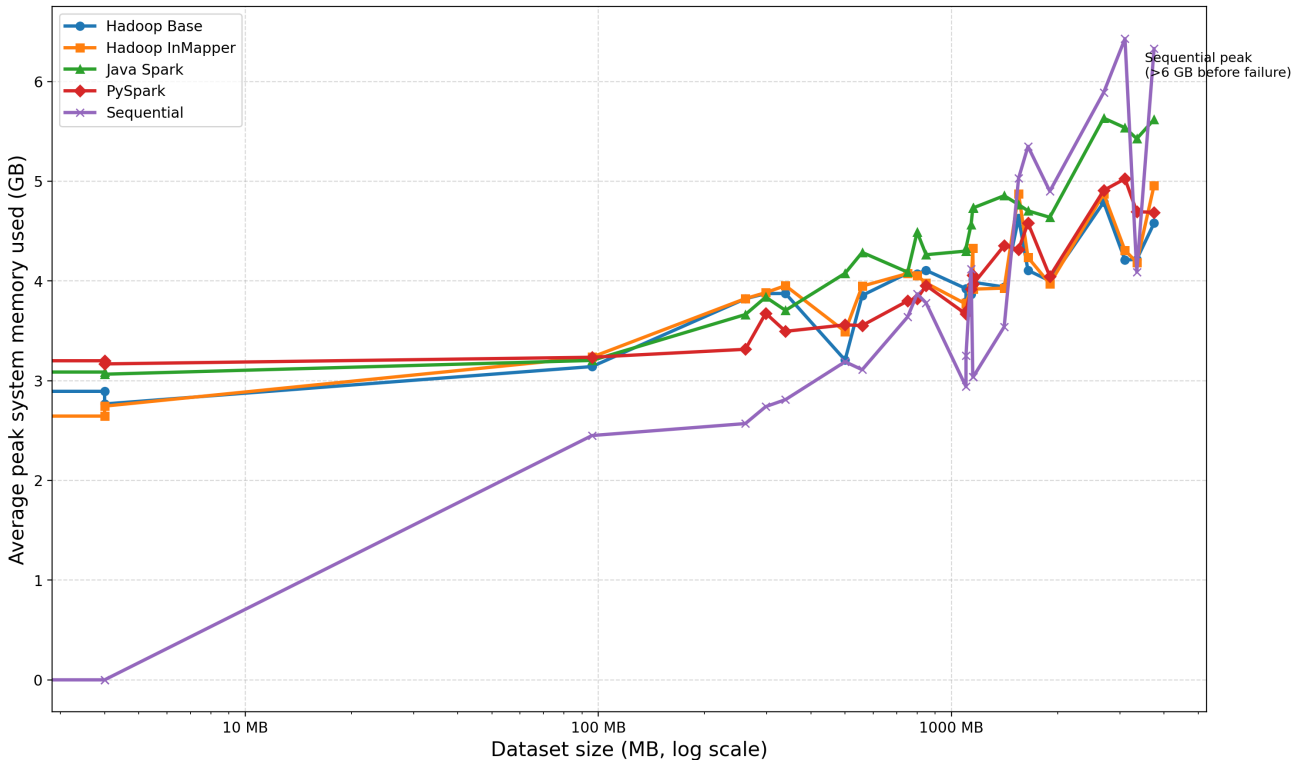


Figure 4 — Peak namenode system memory vs. dataset size (all 25 datasets).

X-axis: dataset size in MB (log scale). Y-axis: `system_used_max_gb`, the peak operating-system memory observed on the `namenode` machine (the node running the client/driver process and one YARN worker), averaged over all parameter values per dataset/method. Data from Table 7. Shows that memory usage on this node scales upward with data size for every method, confirming real, size-driven memory pressure on the coordinating node rather than a flat artefact of the fixed YARN allocation.

Figure 5 — Peak Driver / Client RSS vs Dataset Size

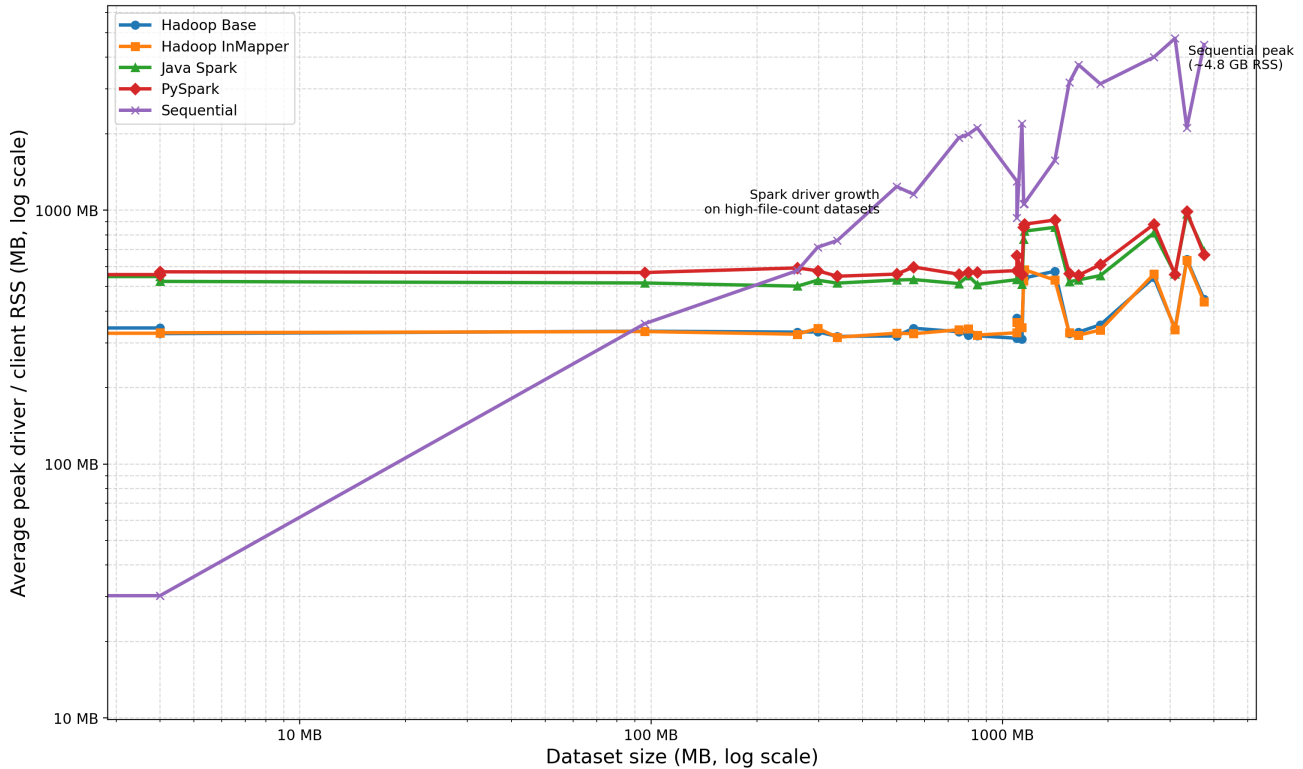


Figure 5 — Peak driver/client RSS vs. dataset size (all 25 datasets).

X-axis: dataset size in MB (log scale). Y-axis: driver RSS in MB (log scale recommended). Data from Table 8. Shows the separation between Hadoop’s flat, low driver memory and both Spark engines’ higher driver memory, with Sequential Python’s near-linear growth shown separately due to its much larger range.

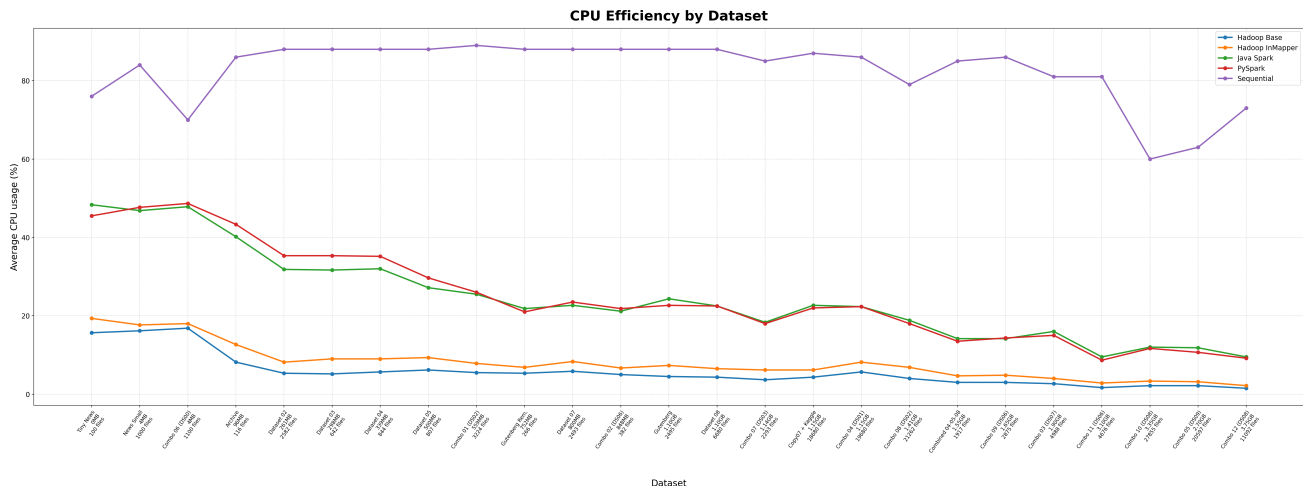


Figure 6 — CPU efficiency of the client/driver process vs. dataset (all 25 datasets).

Y-axis: percentage of CPU time used by the client/driver process during each job (Percent of CPU this job got from `/usr/bin/time -v`), averaged over all parameter values per dataset/method. This metric is collected exclusively from the `namenode` machine and reflects the CPU utilisation of the job-submission and coordination process only — not the total CPU usage across all three cluster nodes. For the distributed methods (Hadoop Base, Hadoop InMapper, PySpark, Java Spark), the actual heavy computation runs inside YARN containers on all three nodes and is not captured here; the namenode driver process handles job planning, progress tracking, and result collection. Sequential Python is the exception: since its entire computation runs in a single process on the namenode, its CPU percentage reflects real computational work rather than coordination overhead.

8 Key Findings Summary

This section answers, point by point, each requirement stated in the project specification, consolidating the headline numbers referenced in the Discussion above.

1. Hadoop vs. Spark comparison (core requirement)

The comparison shows a clear size-dependent crossover:

- **Below ≈ 260 MB:** Sequential Python is fastest overall; among distributed methods, Hadoop InMapper leads. Spark startup overhead is the dominant cost at this scale.
- **260 MB to ≈ 1 GB:** Hadoop InMapper remains the fastest distributed method in most cases, though Java Spark begins to match or narrowly beat it on some datasets (e.g. ds-02 at 261 MB: Java Spark 62 s vs. InMapper 78 s).
- **Above ≈ 1 GB:** Both Spark engines consistently outperform Hadoop InMapper. On the largest dataset (combo-12, 3.84 GB): Java Spark 245 s, PySpark 265 s, InMapper 345 s, Hadoop Base 498 s, Sequential 805 s.

Answer: Hadoop is more efficient at small scale due to lower fixed overhead; Spark overtakes Hadoop as data volume grows, since its in-memory shuffle becomes more efficient than Hadoop’s disk-backed shuffle on this cluster.

2. In-mapper combining (extra requirement)

Hadoop InMapper outperformed Hadoop Base on every one of the 25 datasets, with a mean speedup of $1.53\times$ (range $1.06\times$ – $1.83\times$), growing with dataset size (31% faster at 3.84 GB). Its best time is always achieved at a low reducer count ($r = 1$ – 4 in most cases) — in-mapper combining is more effective than simply adding more reducers.

3. Sequential Python non-parallel baseline (extra requirement)

Fastest method below ≈ 100 MB (zero distributed coordination overhead); falls behind every distributed method above 1 GB; fails entirely at 3.14 GB (combo-11, OOM, exit 137, after 423 s). Its driver RSS grows almost linearly from 13 MB to 4,751 MB, while the Hadoop driver RSS stays flat at 321–640 MB throughout. **Answer:** the non-parallel solution is viable only for small inputs.

4. Effect of increasing reducer / partition count (extra requirement)

- **Hadoop InMapper:** mean elapsed time is minimised at $r = 2$ (129.6 s) and $r = 4$ (130.5 s) across all 25 datasets; $r = 24$ is consistently worst (152.5 s).
- **Spark:** both engines benefit from higher partition counts on large datasets, typically settling between $p = 24$ and $p = 40$, but gains plateau beyond $p = 32$ and occasionally reverse slightly. Memory usage varies by less than 0.5 GB across the full parameter range for a fixed dataset — parallelism affects speed, not memory footprint.

5. Execution time and memory statistics (core requirement)

Collected for all 625 job runs ($5 \text{ methods} \times 6 \text{ parameters} \times 25 \text{ datasets}$): elapsed wall-clock time (Tables 5–6), cluster-wide system memory, and driver-side RSS (Tables 7–8). Java Spark is consistently faster than PySpark at scale (≈ 8 – 33% on large datasets) but uses more system memory (≈ 0.2 – 0.9 GB more), due to native JVM execution eliminating PySpark’s Py4J serialization overhead.

Overall: no single method is universally best. The optimal choice depends on data scale: sequential Python for very small inputs, Hadoop InMapper for small-to-medium distributed workloads, and Spark (Java implementation) for large-scale processing above ≈ 1 GB.

9 Discussion

This section interprets the patterns behind the results reported above; the headline numbers themselves are consolidated once, without repetition, in the Key Findings Summary (Section 8).

Why InMapper beats Base. In-mapper combining eliminates repeated intermediate key-value pairs for the same word within a file before they ever reach the shuffle phase. This directly reduces sort, transfer, and reducer load, and the effect holds at every dataset size tested. Because the map phase already performs most of the aggregation, adding more reducers brings little extra benefit and can add pure scheduling overhead — which is why InMapper’s optimal reducer count is consistently low (Table 6).

Why Spark eventually overtakes Hadoop. On small jobs, Spark’s fixed startup cost (container negotiation and JVM initialisation, roughly 11–22s) is a large fraction of total runtime, so Hadoop InMapper’s lighter-weight job startup wins. As dataset size grows, this fixed cost becomes a smaller share of the total, while Spark’s in-memory, DAG-based shuffle increasingly outperforms Hadoop’s disk-backed shuffle — producing the crossover quantified in Section 8.

Why Java Spark beats PySpark at scale. PySpark’s Py4J bridge and separate Python worker processes add serialization and inter-process communication cost on every record. Java Spark runs natively inside the JVM and avoids this layer entirely, at the cost of holding more data directly in JVM heap — which is consistent with Java Spark’s consistently higher system memory usage in Table 7.

Why memory barely responds to parallelism. Splitting a fixed amount of input data into more partitions or reducers changes how the work is scheduled, not how much data must be held and processed overall. This is consistent with the near-flat memory curves observed across the full parameter sweep for every method (variation under 0.5 GB for a fixed dataset), in contrast to the much larger variation in elapsed time for the same sweep.

Why Sequential Python fails at scale. Unlike the distributed methods, the sequential baseline keeps the entire index as a single in-process Python dictionary with no spill-to-disk mechanism. Its driver RSS therefore grows almost linearly with dataset size (Table 8), eventually exceeding available RAM and triggering the out-of-memory failure described in Section 4.4.

10 Conclusion

We implemented and evaluated five solutions for inverted-index construction across 25 datasets ranging from 0.43 MB to 3.84 GB, including 12 combined datasets created without any physical data duplication on HDFS. All 625 individual jobs were run under a single, fixed cluster configuration; 624 completed successfully, with the one failure (sequential Python at 3.1 GB) being an expected consequence of its single-process, no-spill design rather than an experimental defect.

The project confirms that reducing shuffle volume through in-mapper combining can matter more than framework choice at small to medium scale, but that this advantage is not unconditional: as dataset size and file count grow, the more general-purpose Spark execution engine — particularly its native Java implementation — overtakes a carefully optimised Hadoop in-mapper-combining solution. Memory usage scales measurably with dataset size for every method, while remaining largely insensitive to the reducer/partition parameter, indicating that parallelism in this setup is primarily a lever for speed rather than for memory footprint. A complete breakdown of these findings against each project requirement is given in Section 8.

Appendix: Elapsed Time vs. Parallelism, by Dataset

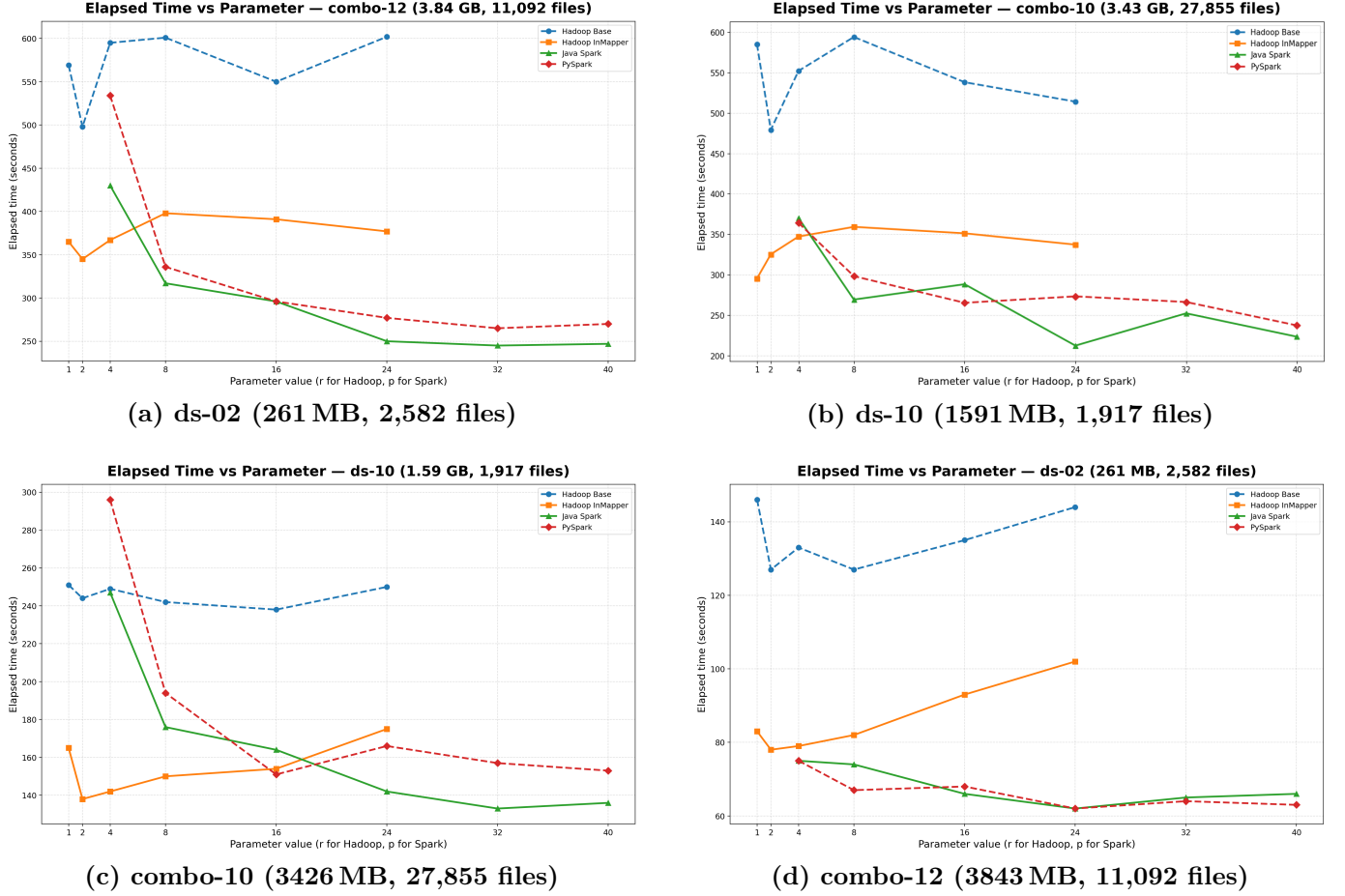


Figure 1: Elapsed time (seconds, Y-axis) vs. parallelism parameter (X-axis: $r \in \{1, 2, 4, 8, 16, 24\}$ for the two Hadoop methods, $p \in \{4, 8, 16, 24, 32, 40\}$ for the two Spark methods), one line per method, for four representative datasets spanning small to large scale and low to high file count. (a) At small scale, fixed Spark startup overhead keeps both Spark engines above Hadoop InMapper across the whole parameter range. (b) At medium scale, Hadoop Base and Hadoop InMapper diverge most clearly in their optimal reducer count ($r = 16$ vs. $r = 2$), while both Spark engines already sit below both Hadoop variants. (c) The highest file-count dataset shows the least regular curves of the four, including Hadoop InMapper’s unusual $r = 1$ optimum and PySpark/Java Spark settling on different optimal partition counts ($p = 40$ vs. $p = 24$). (d) At the largest scale tested, all five methods are most clearly separated, both Spark engines show a visible plateau beyond $p \approx 24$ – 32 , and Java Spark is the fastest configuration overall. For the sequential Python baseline, the corresponding single-run times were 48 s for ds-02, 265 s for ds-10, 840 s for combo-10, and 805 s for combo-12.